Coding Competency Test

1. Foo Bar

Question

Write a simple console program that prints the number from 1 to n, for each number x :

- print "foo", if x is divisible by 3
- print "bar", if x is divisible by 5
- print "foobar", if x is divisible by 3 and 5
- print the number itself, if x satisfies none of the rule Here's a sample output of such program with n=15

> 1, 2, foo, 4, bar, foo, 7, 8, foo, bar, 11, foo, 13, 14, foobar

Answer

We should loop for N,
and each loop we check the rules applied to the number.
to make sure all rules are checked, we do not use `else` in `if` statement
construct the output for each number by those rules.
then consolidate or it to final result
and print this final result

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Input N :");
        string input = Console.ReadLine();
        int number;
        if (!int.TryParse(input, out number))
        {
            Console.WriteLine("Invalid input. Please enter a valid integer.");
            return;
        }

        List<string> results = new List<string>();

        for (int x = 1; x <= number; x++)
        {
            string result = "";

            if (x % 3 == 0) result += "foo";
            if (x % 5 == 0) result += "bar";

            if (string.IsNullOrEmpty(result))
            {
                results.Add(x.ToString());
```

```
            }
            else
            {
                results.Add(result);
            }
        }

        Console.WriteLine(string.Join(", ", results));
    }
}
```

2. Foo Bar Jazz

Question

Continuing on the previous question. Add the following rules :

- print "jazz", if x is divisible by 7

This means for x=21, x=35 and x=105 the program should print "foojazz", "barjazz" and "foobarjazz" respectively.

Answer

Add new rule (jazz) to the validation section
`if (x % 7 == 0) result += "jazz";`

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Input N :");
        string input = Console.ReadLine();
        int number;
        if (!int.TryParse(input, out number))
        {
            Console.WriteLine("Invalid input. Please enter a valid integer.");
            return;
        }

        List<string> results = new List<string>();

        for (int x = 1; x <= number; x++)
        {
            string result = "";

            if (x % 3 == 0) result += "foo";
            if (x % 5 == 0) result += "bar";
            if (x % 7 == 0) result += "jazz";

            if (string.IsNullOrEmpty(result))
```

```
            {
                results.Add(x.ToString());
            }
            else
            {
                results.Add(result);
            }
        }

        Console.WriteLine(string.Join(", ", results));
    }
}
```

3. Foo Baz Bar Jazz Huzz

Question

Continuing on the previous question. Using the same divisible logic, use the table below as the rules :

- 3 : "foo"
- 4 : "baz"
- 5 : "bar"
- 7 : "jazz"
- 9 : "huzz"

Answer

Add new rules to the validation section

```
if (x % 3 == 0) result += "foo";
if (x % 4 == 0) result += "baz";
if (x % 5 == 0) result += "bar";
if (x % 7 == 0) result += "jazz";
if (x % 9 == 0) result += "huzz";
```

so the final code will be

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Input N :");
        string input = Console.ReadLine();
        int number;
        if (!int.TryParse(input, out number))
        {
            Console.WriteLine("Invalid input. Please enter a valid integer.");
            return;
        }
```

```
        List<string> results = new List<string>();

        for (int x = 1; x <= number; x++)
        {
            string result = "";

            if (x % 3 == 0) result += "foo";
            if (x % 4 == 0) result += "baz";
            if (x % 5 == 0) result += "bar";
            if (x % 7 == 0) result += "jazz";
            if (x % 9 == 0) result += "huzz";

            if (string.IsNullOrEmpty(result))
            {
                results.Add(x.ToString());
            }
            else
            {
                results.Add(result);
            }
        }

        Console.WriteLine(string.Join(", ", results));
    }
}
```

4. Dynamic Foo Bar

Question

Turn the generator logic in the code you have so far into a class object and make it so that the client code can configure its own rules i.e. add the following API

> myClass.AddRule(int input, string output)

Answer

Create new Class GeneratorLogic.cs that handle :

- CRUD rules (AddRule -> add or update, RemoveRule, CleanRules, GetRule)
- Perform the logic as we have before in previous question (Generate + Print)

so the new class GeneratorLogic.cs will be :

```
class GeneratorLogic
{
    private readonly SortedDictionary<int, string> _rules = new
SortedDictionary<int, string>();

    public void AddRule(int divisor, string output)
```

```
    {
        if (divisor <= 0)
        {
            throw new ArgumentException("Divisor must be a positive integer.");
        }
        if (string.IsNullOrEmpty(output))
        {
            throw new ArgumentException("Output cannot be null or empty.");
        }
        _rules[divisor] = output;
    }

    public SortedDictionary<int, string> GetRules()
    {
        return _rules;
    }

    public void RemoveRule(int divisor)
    {
        if (!_rules.ContainsKey(divisor))
        {
            throw new KeyNotFoundException("Divisor not found in rules.");
        }
        _rules.Remove(divisor);
    }

    public void ClearRules()
    {
        _rules.Clear();
    }

    public List<string> Generate(int number)
    {
        if (number <= 0)
        {
            throw new ArgumentException("Number must be a positive integer.");
        }

        List<string> results = new List<string>();

        for (int x = 1; x <= number; x++)
        {
            string result = "";

            foreach (var rule in _rules)
            {
                if (x % rule.Key == 0)
                {
                    result += rule.Value;
                }
            }

            if (string.IsNullOrEmpty(result))
            {
```

```
                results.Add(x.ToString());
            }
            else
            {
                results.Add(result);
            }
        }

        return results;
    }

    public void PrintResults(List<string> results)
    {
        Console.WriteLine(string.Join(", ", results));
    }
}
```

and by this, client can configure its own rules. this is the example :

```
class Program
{
    static void Main(string[] args)
    {
        GeneratorLogic generator = new GeneratorLogic();
        generator.AddRule(3, "foo");
        generator.AddRule(4, "baz");
        Console.WriteLine("Generated Rules:");
        foreach (var rule in generator.GetRules())
        {
            Console.WriteLine($"Divisor: {rule.Key}, Output: {rule.Value}");
        }
        Console.WriteLine("Input N :");
        string input = Console.ReadLine();
        int number;
        if (!int.TryParse(input, out number))
        {
            Console.WriteLine("Invalid input. Please enter a valid integer.");
            return;
        }

        List<string> results = generator.Generate(number);
        generator.PrintResults(results);
    }
}
```